CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G2
System Integration and Architecture

System Design Document (SDD)

Project Title: LaundryLink

Prepared By: Cañete, Rod Gabrielle Maluya

Version: 2.0

Date: March 2, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	Feb 23, 2026	Cañete, Rod Gabrielle Maluya	**Executive Summary:** Project overview, objectives (8 items), and full scope (included/excluded features) all updated to reflect mandatory integrations **Introduction**: Purpose statement updated to reference all IT342 mandatory features **Functional Requirements Specification**: User journeys updated, MoSCoW corrected, 10 detailed feature specs written, 7 acceptance criteria in Given/When/And/Then format **Non-Functional Requirements**: Performance, security (BCrypt, DTOs, no plain-text passwords), compatibility, and usability requirements documented	Draft
2.0	Mar 2, 2026	Cañete, Rod Gabrielle Maluya	**Section 4.1 Component Diagram:** Added full component diagram illustrating the three-tier architecture — ReactJS web and Kotlin Android clients, Spring Boot backend with all five layers (Security, Controller, DTO, Service, Repository), Supabase PostgreSQL database, and all external integrations (Google OAuth 2.0, PayMongo, SMTP, Supabase Storage, Google Maps API, QR Code generation) **Section 6.1 Entity Relationship Diagram:** Replaced placeholder e-commerce schema with the correct LaundryLink-specific ERD in Crow's Foot notation. Covers 7 entities: users, shops, services, slot_configs, bookings, payments, and refresh_tokens. Includes all primary keys, foreign keys, unique constraints, data types, and 8 annotated	Final

			relationships reflecting the actual booking lifecycle and system logic. **Section 7.0 UI/UX Design:** Finalized all wireframes for both web and mobile applications. Web wireframes completed for: Homepage/Shop Listing (with embedded Google Map and shop cards), Login & Register page (email/password form and Google OAuth button), Booking Flow (4-step process: date picker, time slot selector, file upload, booking summary), Payment & Confirmation page (PayMongo redirect, QR code display), My Bookings page (status lifecycle badges), and Admin Dashboard (booking table, status update panel, slot limit configuration). Mobile wireframes completed for: Home Screen, Booking Flow Screen, My Bookings Screen, and Admin Screens — all following the 44×44px touch target minimum and bottom navigation pattern.	

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 5

1.0 INTRODUCTION 7

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 7

3.0 NON-FUNCTIONAL REQUIREMENTS 14

4.0 SYSTEM ARCHITECTURE 15

5.0 API CONTRACT & COMMUNICATION 17

6.0 DATABASE DESIGN 30

7.0 UI/UX DESIGN 29

8.0 PLAN 46

EXECUTIVE SUMMARY

1.1 Project Overview

LaundryLink is a prepaid priority queue management system for laundry shops. It allows customers to reserve limited processing slots at selected partner shops, pay online via PayMongo, and receive a QR-coded booking confirmation. The system is built with a Spring Boot backend, ReactJS web frontend, and a Kotlin Android mobile app, all connected to a Supabase (PostgreSQL) database.

1.2 Objectives

1. Develop a functional laundry slot reservation system with JWT-based authentication, booking logic, and online payment via PayMongo (sandbox mode).
2. Implement a three-tier architecture using Spring Boot (backend), ReactJS (web), and Kotlin (mobile).
3. Integrate Google OAuth 2.0 login in addition to traditional email/password registration.
4. Consume an external public API (e.g., OpenWeatherMap) to display meaningful weather/environmental information on the shop selection page.
5. Implement file upload functionality for customers to attach laundry-specific photos or special instruction documents to a booking.
6. Send transactional and notification emails via SMTP (e.g., welcome email on registration, booking confirmation receipt).
7. Enforce daily slot capacity limits and manage a structured booking lifecycle: Pending Payment → Paid → Dropped Off → Processing → Completed.
8. Generate QR code booking confirmations and provide an admin panel for status management and slot limit configuration.

1.3 Scope

Included Features:

- User registration and authentication (email/password) with JWT
- Google OAuth 2.0 social login with JWT generation
- /me endpoint to retrieve the current authenticated user
- Role-based access control: CUSTOMER and ADMIN roles
- Shop selection from a curated list of 3–5 partner laundry shops
- Service tier selection: Standard (regular queue) and Priority (limited daily slots, higher price)
- Date and time slot booking with real-time capacity enforcement
- File upload: customers can attach laundry photos or special instruction files (PDF/image) to a booking
- External API integration: OpenWeatherMap API displayed on shop listing page
- PayMongo Payment Intent integration (sandbox) with webhook confirmation
- SMTP email notifications: welcome email on registration, booking receipt on payment
- QR code generation for booking confirmation
- Admin panel: view/filter bookings, update status, configure daily slot limits
- Kotlin Android mobile application (full feature parity with web)
- Supabase (PostgreSQL) database with relational schema (7+ tables)

Excluded Features:

- Delivery or rider logistics
- Real-time GPS tracking
- Reviews and ratings
- In-app messaging
- Loyalty or subscription systems
- Multi-vendor self-registration
- Analytics dashboards, revenue graphs, or dynamic pricing

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for LaundryLink. It details the functional and non-functional requirements, system architecture, API contracts, database schema, UI/UX wireframes, and a 10-week implementation plan. All mandatory features per IT342 Final Defense requirements are covered, including authentication, RBAC, external API integration, Google OAuth, file upload, PayMongo payment gateway, and SMTP email sending.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: LaundryLink
Domain: Laundry Service / Queue Management
Primary Users: Laundry Customers and Shop Administrators

Problem Statement: Traditional laundry shops rely on first-come, first-served queues and manual payments. During peak hours, customers face long wait times and unpredictable turnaround.
Solution: A prepaid online booking system that reserves priority processing slots, enforces capacity limits, and integrates PayMongo for secure online payment — giving customers faster, predictable laundry service.

2.2 Core User Journeys

Journey 1: First-time Customer Booking

1. Customer visits LaundryLink web or opens mobile app.
2. Clicks "Sign Up" and creates an account OR logs in via Google OAuth.
3. Receives a welcome email via SMTP upon successful registration.
4. Browses the list of partner laundry shops
5. Selects a shop and views available service tiers.
6. Picks an available date and time slot.
7. Reviews booking summary and proceeds to PayMongo payment (sandbox).
8. Receives booking confirmation with QR code and an email receipt via SMTP.

Journey 2: Returning Customer Booking

1. Customer logs in with existing credentials OR via Google OAuth
2. Selects preferred partner shop
3. Chooses Priority service tier

4. Checks slot availability for desired date
5. Completes PayMongo prepaid payment
6. Downloads QR code and presents it at the shop on arrival

Journey 3: Administrator Booking Management

1. Admin logs into the admin panel (web or mobile) with admin JWT.
2. Views all bookings filtered by date and shop
3. Updates booking statuses (Dropped Off, Processing, Completed)
4. Adjusts the daily Priority slot limit for a specific shop
5. Monitors slot utilization to prevent overbooking
6. Reviews completed bookings for records

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout) with JWT session handling and /me endpoint.
2. Google OAuth 2.0 login with JWT generation (must generate own JWT after OAuth).
3. Role-based access: CUSTOMER and ADMIN roles enforced at both API and UI levels.
4. Shop listing with service tier display (Standard and Priority).
5. Date and time slot selection with capacity enforcement.
6. External API integration: GoogleMap data displayed on shop listing page.
7. PayMongo Payment Intent integration (sandbox) with webhook confirmation.
8. SMTP email sending: welcome email on registration, receipt on booking confirmation.
9. QR code generation for booking confirmation.
10. Admin panel: view/filter bookings, update status, set slot limits.

SHOULD HAVE

1. User booking history page.
2. Booking status visibility for customers.
3. Form validation with clear error messages.
4. Responsive design across mobile and desktop.

COULD HAVE

1. Real-time status updates via WebSocket or polling (bonus feature).
2. Booking cancellation (before payment confirmation).
3. Basic slot utilization summary for admin.

WON'T HAVE

1. Real-time GPS tracking.
2. Reviews and ratings.
3. In-app messaging or push notifications.
4. Loyalty systems, subscriptions, or multi-vendor self-registration.
5. Dynamic pricing or analytics dashboards.

2.4 Detailed Feature Specifications

Feature 1: User Authentication & Security

- Screens: Registration, Login, Forgot Password
- Fields: Email, Password, Confirm Password, Full Name
- Validation: Email format, minimum 8-character password, unique email check
- Security: JWT access tokens, BCrypt password hashing (salt rounds = 12), protected routes
- API Endpoints: POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me

Feature 2: Google OAuth 2.0 Login

- Users can click "Login with Google" on web or mobile.
- After OAuth callback, the backend links/creates a user record in the database.
- Backend generates its own JWT (not Google's token) and returns it to the client.
- If the user is new, a welcome email is sent automatically via SMTP.
- API Endpoints: GET /api/auth/oauth2/google, GET /api/auth/oauth2/callback

Feature 3: Role-Based Access Control

- Roles: CUSTOMER and ADMIN.
- ADMIN role: access to admin endpoints, booking management, slot limit configuration.
- CUSTOMER role: access to shop listing, booking creation, own booking history.

• API-level enforcement: Spring Security @PreAuthorize annotations on admin endpoints.

• UI-level enforcement: React route guards and Kotlin screen navigation guards.

Feature 4: Shop & Service Selection

• Screens: Shop List, Shop Detail with Service Tiers

• Display: Shop name, address, operating hours, available service tiers, local weather (from OpenWeatherMap API)

• Service Tiers: Standard (regular queue, lower price), Priority (limited daily slots, higher price)

• API Endpoints: GET /api/shops, GET /api/shops/{id}, GET /api/shops/{id}/services

Feature 5: External API Integration — Google Maps Platform

• Consumes the Google Maps Platform APIs
• Uses the Google Places API to dynamically retrieve nearby laundry shops based on the user´s current location.
• Displays real shop data (name, address, rating, and location pins) on an interactive map within the web and mobile applications.
• Backend optionally validates and stores selected shop data (e.g., place_id, name, address) and exposes it via `GET /api/shops/nearby?lat={latitude}&lng={longitude}`.
• Meaningful use case: customers can easily discover and select nearby laundry shops for booking without manually searching, improving accessibility and user experience.

Feature 6: Booking & Slot Management

• Screens: Date/Time Slot Selector, Booking Summary
• Functions: Check available slots, reserve slot, create Pending Payment booking
• Enforcement: System rejects booking when daily Priority slot limit is reached
• API Endpoints: GET /api/slots?shopId={id}&serviceId={id}&date={date}, POST /api/bookings, GET /api/bookings/{id}, GET /api/bookings/my (own bookings)

Feature 7: File Upload

• Customers can attach a laundry photo or PDF instructions to their booking.

• File stored on server filesystem or cloud storage (e.g., Supabase Storage).

• File path/URL linked to the booking record in the database.

• Admin can view/download the attached file from the booking detail panel.

• Customer can also view/download their own uploaded file.

• API Endpoints: POST /api/bookings/{id}/upload (multipart/form-data), GET /api/bookings/{id}/file

Feature 8: Payment Integration — PayMongo (Sandbox)

• Process: Create Payment Intent → Redirect to PayMongo (sandbox) → Webhook fires on payment success → Booking status updated to Paid → QR code generated.

• Validate webhook signature, update booking, generate QR code URL.

• Payment results (payment intent ID, status, amount) recorded in the payments table.

• Handles both success and failure cases.

• No simulated payments — uses real PayMongo test mode keys.

• API Endpoints: POST /api/payments/create-intent, POST /api/payments/webhook

Feature 9: SMTP Email Sending

- Email 1 (Account-related): Welcome email sent to new users upon registration or first Google OAuth login. Contains their name and a link to start booking.
- Email 2 (System notification): Booking receipt email sent after PayMongo confirms payment. Contains booking ID, shop, date, time, and QR code image.
- Uses JavaMailSender (Spring Boot) with SMTP configuration (e.g., Gmail SMTP or SendGrid).
- Console print is NOT used — emails are genuinely delivered to the user's inbox.

Feature 10: Admin Panel

- Screens: Booking List (filterable by date/shop), Status Update Panel, Slot Limit Config, File Viewer

- Functions: View all bookings, filter by date, mark status (Dropped Off, Processing, Completed), set daily Priority slot limit per shop, view/download uploaded files
- Access Control: JWT with ADMIN role required
- API Endpoints: GET /api/admin/bookings, PATCH /api/admin/bookings/{id}/status, PUT /api/admin/shops/{id}/slot-limit

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new customer
- When I enter a valid email and a password of at least 8 characters
- And the confirm password matches
- And I click "Create Account"
- Then my account should be created in the database
- And I should receive a JWT and be automatically logged in
- And a welcome email should be delivered to my inbox
- And I should be redirected to the shop listing page

AC-2: Google OAuth Login

- Given I am on the login page
- When I click "Login with Google"
- And I complete the Google consent flow
- Then my account should be created or linked in the database
- And the backend should generate its own JWT (not Google's token)
- And I should be redirected to the shop listing page
- And if this is my first login, a welcome email should be sent to my email address

AC-3: Google Maps Shop Locations Displayed

- Given I am on the shop listing page
- When the page loads
- Then an interactive Google Map should be rendered on the page
- And each partner shop should appear as a clickable marker pin on the map
- And clicking a marker should show the shop name, address, and a "Book Now" link
- And the map data should be sourced from the Google Maps API (not hardcoded or mocked)

AC-4: Successful Booking with File Upload

- Given I am logged in as a customer

- When I complete the booking flow and optionally attach a laundry photo or PDF
- And I click "Proceed to Payment"
- Then the uploaded file should be saved on the server
- And the file URL should be linked to my booking record in the database
- And I should be able to view or download the file from My Bookings
- And the admin should also be able to view or download the file from the admin panel

AC-5: Successful Payment and Email Receipt

- Given I have a booking with status "Pending Payment"
- When I am redirected to the PayMongo sandbox payment page
- And I complete the payment using test card credentials
- And PayMongo sends the webhook to LaundryLink
- Then the webhook signature should be validated
- And my booking status should be updated to "Paid"
- And a QR code should be generated and displayed on the confirmation screen
- And a receipt email should be delivered to my inbox with booking details
- And the payment record should be saved in the payments table

AC-6: Admin Booking Status Update

- Given I am logged in as an administrator
- When I view the booking list in the admin panel
- And I select a booking with status "Paid"
- And I update the status to "Dropped Off"
- Then the booking record in the database should reflect the new status
- And the customer should see the updated status on their My Bookings page

AC-7: Role-Based Access Control

- Given I am logged in as a CUSTOMER
- When I attempt to access an admin-only endpoint (e.g., GET /api/admin/bookings)
- Then I should receive a 403 Forbidden response
- And the admin panel pages should not be accessible or visible in the UI
- And given I am logged in as an ADMIN
- Then I should have full access to all admin endpoints and panels

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication with BCrypt password hashing (salt rounds = 12)
- No plain-text passwords stored in the database
- SQL injection prevention via parameterized queries (Spring Data JPA)
- XSS protection on all user inputs
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require ADMIN role verification at the API level
- DTOs used on all API responses — passwords never exposed

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** Android API Level 26+ (Android 8.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

- Complete first booking within 5 minutes for new users
- WCAG 2.1 Level AA compliance for web interface
- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile
- Keyboard navigation support

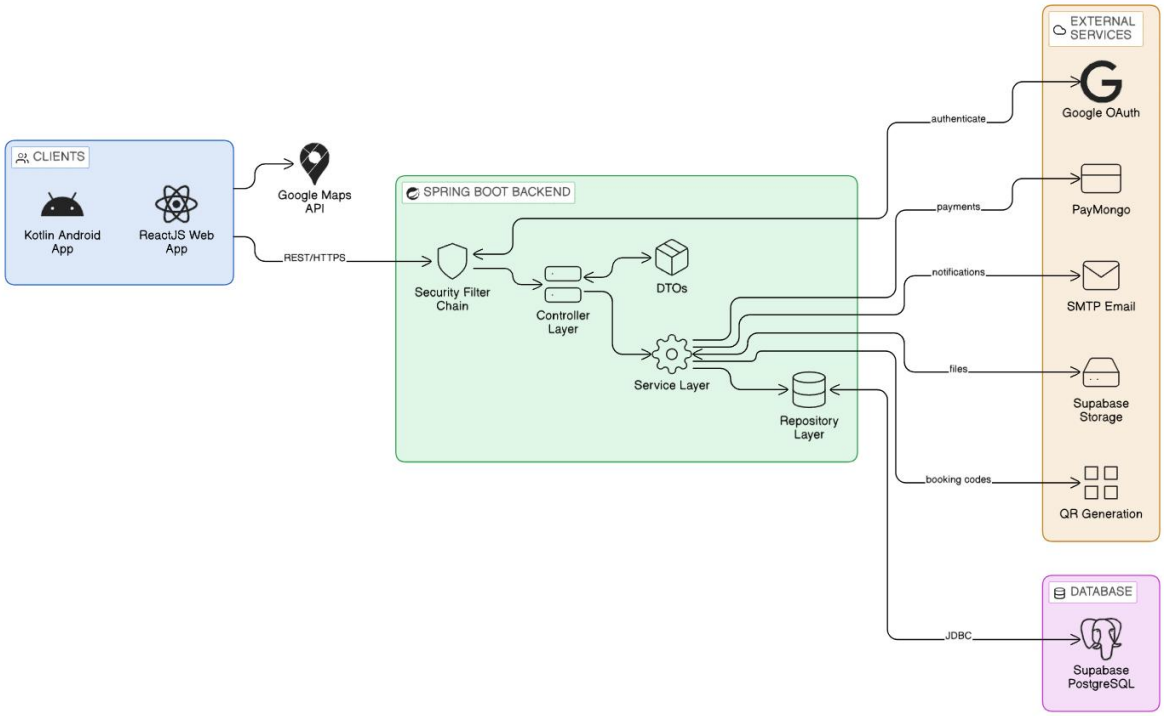
4.0 SYSTEM ARCHITECTURE

LaundryLink follows a Layered (N-Tier) Architecture pattern on the backend. This pattern organizes code into distinct layers where each layer has a specific responsibility and only communicates with the layer directly below it. This enforces separation of concerns, makes the codebase testable, and simplifies debugging.

Backend Layers

- Controller Layer: Handles incoming HTTP requests, validates input at the API boundary, delegates to the Service layer, and returns DTOs. Example: BookingController, AuthController.
- Service Layer: Contains all business logic — slot capacity checks, payment webhook processing, QR code generation, email sending. Example: BookingService, PaymentService, EmailService.
- Repository Layer: Manages all database access via Spring Data JPA repositories. Example: BookingRepository, UserRepository.
- DTOs (Data Transfer Objects): Used on all API inputs and outputs. Passwords and sensitive fields are never exposed in responses.
- Security Configuration: Spring Security filter chain, JWT filter, OAuth2 configuration, role-based access rules.
- Global Exception Handler: @ControllerAdvice catches all exceptions and returns standardized error JSON responses with proper HTTP status codes.

4.1 Component Diagram



Technology Stack:

Category	Technology
Backend	Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA, Spring Mail
Database	Supabase (hosted PostgreSQL)
Authentication	JWT (JJWT library), BCrypt, Google OAuth 2.0
Payment Gateway	PayMongo API (Sandbox/Test Mode)
Email	JavaMailSender + SMTP (Gmail / SendGrid)
External API	Google Maps JavaScript API + Geocoding API
File Storage	Supabase Storage or server filesystem
Web Frontend	ReactJS 18, JavaScript, Tailwind CSS, Axios

Mobile	Kotlin, Jetpack Compose, Retrofit2
Build Tools	Maven (Backend), npm (Web), Gradle (Android)
Deployment	Railway or Render (Backend), Vercel (Web), APK sideload (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Field	Details
Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	{ "success": boolean, "data": object|null, "error": { "code": string, "message": string, "details": object|null }, "timestamp": string }

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Field	Details
Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": <email>, "password": <password>, "firstname": <firstname>, "lastname": <lastname> }
Response Structure	{ "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, }, "accessToken": <token>, "refreshToken": <token> } }

User Login

Field	Details

Description	User Login
API URL	/auth/login
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": <email>, "password": <password> }
Response Structure	{ "success": boolean, "data": { user { "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role> }, "accessToken": <token>, "refreshToken": <token> } }

Logout

Field	Details
Description	Logout current authenticated user
API URL	/auth/logout
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	{ "success": boolean, "data": { "message": "Logged out successfully" } }

Get Current Authenticated User

Field	Details
Description	Returns the profile of the currently authenticated user
API URL	/auth/me
HTTP Request Method	GET
Format	JSON for all requests/responses

Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	{ "success": boolean, "data": { user { "id": <id>, "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role> } } }

Google OAuth 2.0 Login

Field	Details
Description	Initiates Google OAuth 2.0 login flow; on callback, creates or links user and returns a system-generated JWT
API URL (Initiate)	/auth/oauth2/google
API URL (Callback)	/auth/oauth2/callback
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": boolean, "data": { user { "id": <id>, "email": <email>, "firstname": <firstname>, "lastname": <lastname>, "role": <role> }, "accessToken": <token> } }

Get All Shops

Field	Details
Description	Returns a list of all partner laundry shops
API URL	/shops
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None

Response Structure	{ "success": boolean, "data": { "shops": [{ "id": <id>, "name": <name>, "address": <address>, "city": <city>, "operatingHours": <hours> }] } }

Get Shop Details

Field	Details
Description	Returns the details of a specific partner shop
API URL	/shops/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": boolean, "data": { "id": <id>, "name": <name>, "address": <address>, "city": <city>, "operatingHours": <hours> } }

Get Shop Services

Field	Details
Description	Returns the available service tiers for a specific shop
API URL	/shops/{id}/services
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": boolean, "data": { "services": [{ "id": <id>, "name": <name>, "type": "STANDARD" | "PRIORITY", "price": <price> }] } }

Get Available Time Slots

Field	Details

Description	Returns available time slots for a shop, service, and date
API URL	/slots?shopId=<id>&serviceId=<id>&date=<date>
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None (query parameters)
Response Structure	{ "success": boolean, "data": { "availableSlots": ["09:00", "10:00", ...], "slotsRemaining": <count> } }

Get Shop Map Data (Google Maps)

Field	Details
Description	Returns coordinates and metadata for all partner shops to render Google Maps markers on the shop listing page
API URL	/shops/map-data
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": boolean, "data": { "shops": [{ "id": <id>, "name": <name>, "address": <address>, "latitude": <lat>, "longitude": <lng> }] } }

Create Booking

Field	Details
Description	Creates a new booking with status "Pending Payment"
API URL	/bookings
HTTP Request Method	POST

Format	JSON for all requests/responses
Authentication	Bearer token (JWT) — CUSTOMER role
Request Payload	{ "shopId": <id>, "serviceId": <id>, "date": <date>, "timeSlot": <time> }
Response Structure	{ "success": boolean, "data": { "id": <id>, "bookingCode": <code>, "status": "Pending Payment", "date": <date>, "timeSlot": <time> } }

Get Booking by ID

Field	Details
Description	Returns the details of a specific booking including QR code and file URL
API URL	/bookings/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	{ "success": boolean, "data": { "id": <id>, "bookingCode": <code>, "status": <status>, "qrCodeUrl": <url>, "fileUrl": <url>, "shop": { "id": <id>, "name": <n> }, "service": { "id": <id>, "name": <n> }, "date": <date>, "timeSlot": <time> } }

Get My Bookings

Field	Details
Description	Returns all bookings belonging to the currently authenticated customer
API URL	/bookings/my
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) — CUSTOMER role
Request Payload	None

Response Structure	{ "success": boolean, "data": { "bookings": [{ "id": <id>, "bookingCode": <code>, "status": <status>, "date": <date>, "timeSlot": <time> }] } }

Upload File to Booking

Field	Details
Description	Uploads a laundry photo or PDF instructions file and links it to a booking record
API URL	/bookings/{id}/upload
HTTP Request Method	POST
Format	multipart/form-data
Authentication	Bearer token (JWT) — CUSTOMER role
Request Payload	Form field: file (image/pdf, max 10MB)
Response Structure	{ "success": boolean, "data": { "bookingId": <id>, "fileUrl": <url> } }

View / Download Booking File

Field	Details
Description	Returns or redirects to the uploaded file linked to a booking
API URL	/bookings/{id}/file
HTTP Request Method	GET
Format	File stream or redirect URL
Authentication	Bearer token (JWT)
Request Payload	None
Response Structure	File download stream or signed URL redirect

Create Payment Intent

Field	Details

Description	Creates a PayMongo Payment Intent for a booking and returns the checkout URL
API URL	/payments/create-intent
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) — CUSTOMER role
Request Payload	{ "bookingId": <id> }
Response Structure	{ "success": boolean, "data": { "paymentUrl": <url>, "paymentIntentId": <id> } }

PayMongo Webhook

Field	Details
Description	Receives PayMongo webhook events; validates signature, updates booking to Paid, generates QR code, and sends receipt email
API URL	/payments/webhook
HTTP Request Method	POST
Format	JSON (PayMongo signed payload)
Authentication	PayMongo webhook signature header
Request Payload	PayMongo signed webhook payload (handled internally)
Response Structure	{ "success": boolean, "data": { "received": true } }

Get All Bookings (Admin)

Field	Details
Description	Returns all bookings across all customers, filterable by date and shop
API URL	/admin/bookings?date=<date>&shopId=<id>
HTTP Request Method	GET
Format	JSON for all requests/responses

Authentication	Bearer token (JWT) — ADMIN role
Request Payload	None (query parameters)
Response Structure	{ "success": boolean, "data": { "bookings": [{ "id": <id>, "bookingCode": <code>, "customer": { "name": <n>, "email": <email> }, "shop": { "name": <n> }, "service": { "name": <n> }, "date": <date>, "timeSlot": <time>, "status": <status>, "fileUrl": <url> }] } }

Update Booking Status (Admin)

Field	Details
Description	Updates the status of a specific booking
API URL	/admin/bookings/{id}/status
HTTP Request Method	PATCH
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) — ADMIN role
Request Payload	{ "status": "Dropped Off" | "Processing" | "Completed" }
Response Structure	{ "success": boolean, "data": { "id": <id>, "status": <status>, "updatedAt": <timestamp> } }

Set Daily Slot Limit (Admin)

Field	Details
Description	Sets or updates the daily Priority slot limit for a specific shop
API URL	/admin/shops/{id}/slot-limit
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) — ADMIN role
Request Payload	{ "date": <date>, "maxSlots": <count> }
Response Structure	{ "success": boolean, "data": { "updated": true, "slotLimit": <count> } }

5.3 Error Handling

HTTP Status Codes

Code	Meaning
200 OK	Successful request
201 Created	Resource successfully created
400 Bad Request	Invalid input or validation failure
401 Unauthorized	Authentication required or token expired
403 Forbidden	Insufficient permissions (wrong role)
404 Not Found	Resource does not exist
409 Conflict	Duplicate resource (e.g., email already registered)
500 Internal Server Error	Unhandled server error

Error Code Examples

```json
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid email or password",
    "details": "Check your email and password and try again"
  },
  "timestamp": "2026-02-22T09:00:00Z"
}
```

{
  "success": false,
  "data": null,
  "error": {
    "code": "SLOT-001",
    "message": "No slots available",
    "details": {
      "date": "2026-02-22",
      "reason": "Daily Priority slot limit has been reached for this shop"
    }
  },
  "timestamp": "2026-02-22T09:00:00Z"
}

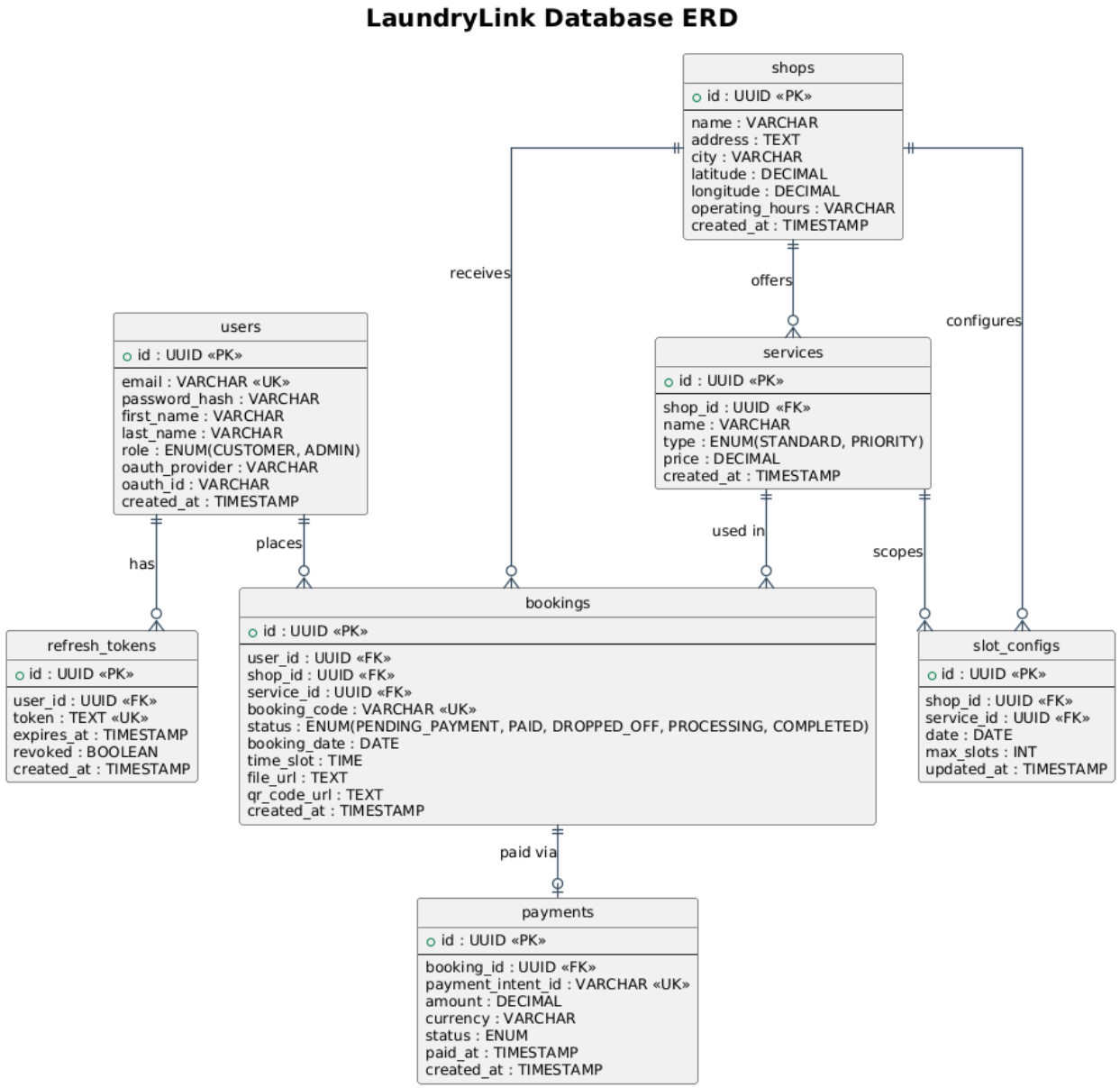
Common Error Codes

Code	Message
AUTH-001	Invalid email or password
AUTH-002	JWT token expired
AUTH-003	Admin role required for this endpoint
OAUTH-001	Google OAuth authentication failed
SLOT-001	No available slots for selected date and time
SLOT-002	Daily Priority slot limit reached
BOOKING-001	Booking not found
UPLOAD-001	File type not allowed (only image/pdf accepted)
UPLOAD-002	File size exceeds 10MB limit
PAYMENT-001	PayMongo Payment Intent creation failed
PAYMENT-002	Webhook signature verification failed
MAPS-001	Google Maps API key invalid or quota exceeded
SYSTEM-001	Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

LaundryLink uses a PostgreSQL database (hosted on Supabase) with the following core tables.



Detailed Relationships:

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)
- **One-to-Many:** Cart → CartItems (Cart contains multiple items)
- **One-to-Many:** Order → OrderItems (Order contains multiple items)
- **Many-to-One:** CartItems → Product (Items reference products)
- **Many-to-One:** OrderItems → Product (Items reference products)

Key Tables:

1. **users** - User accounts and authentication
2. **products** - Product catalog information
3. **carts** - Shopping cart per user
4. **cart_items** - Items in shopping cart
5. **orders** - Customer orders
6. **order_items** - Items in each order
7. **refresh_tokens** - JWT refresh tokens

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at
- **products:** id, name, description, price, stock, image_url, category
- **carts:** id, user_id, created_at
- **cart_items:** id, cart_id, product_id, quantity
- **orders:** id, order_number, user_id, total, status, shipping_address
- **order_items:** id, order_id, product_id, product_name, quantity, price

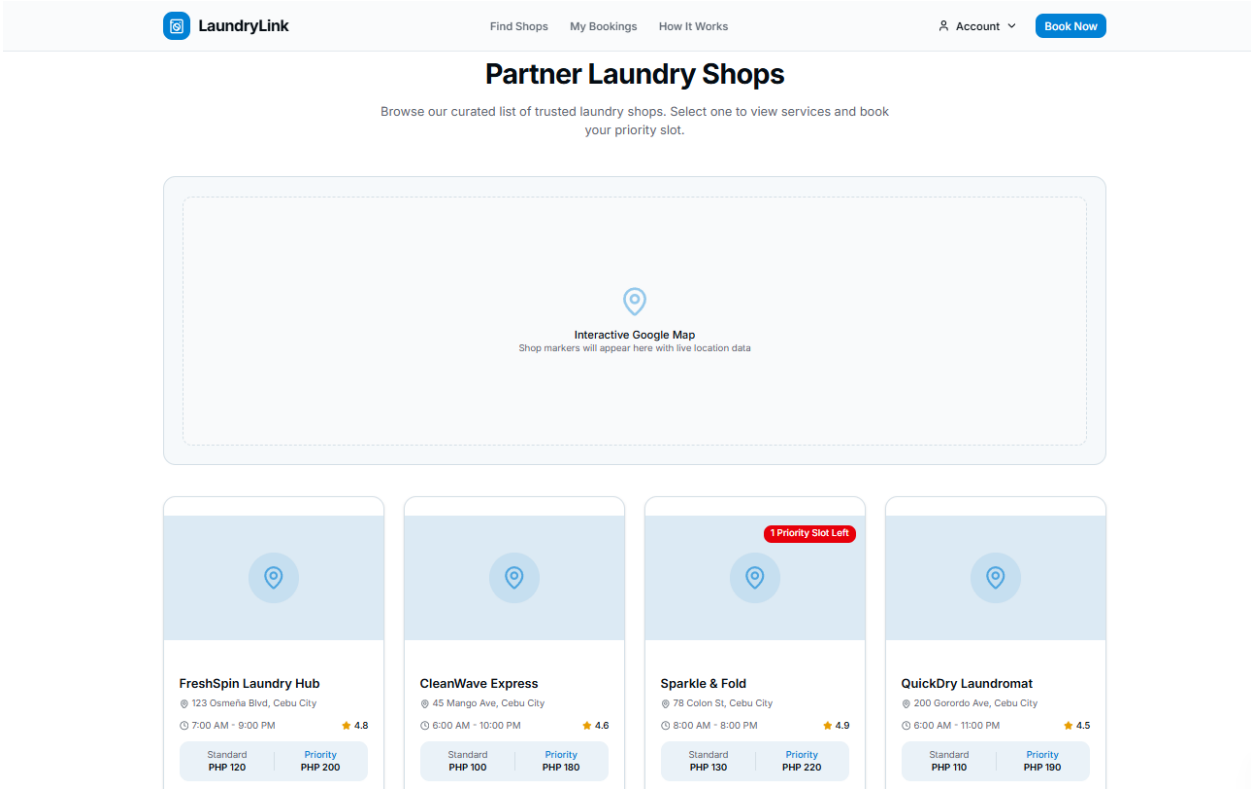
7.0 UI/UX DESIGN

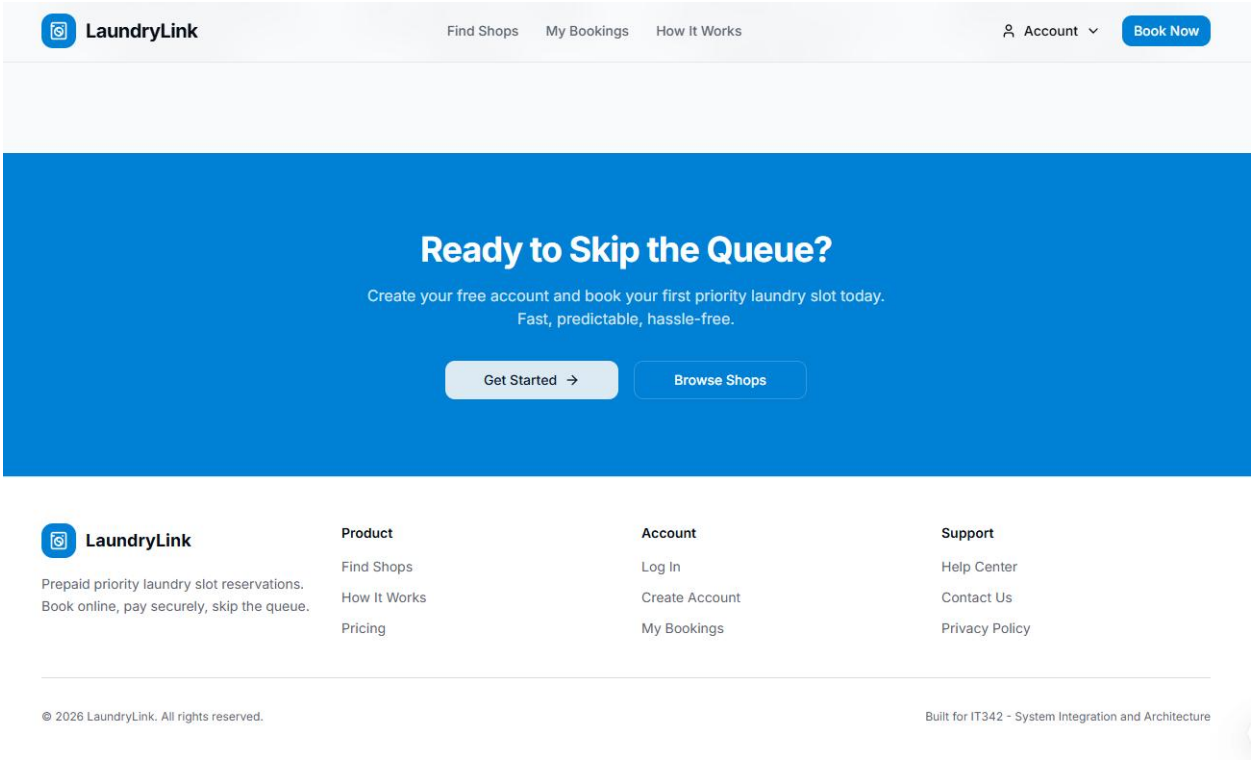
7.1 Web Application Wireframes

https://www.figma.com/make/wX7ZCQzbscXzy42h31IzhW/Design-LaundryLink-Web-App?t=xxOxKdgEHm2QYiOr-1

Homepage / Shop Selection

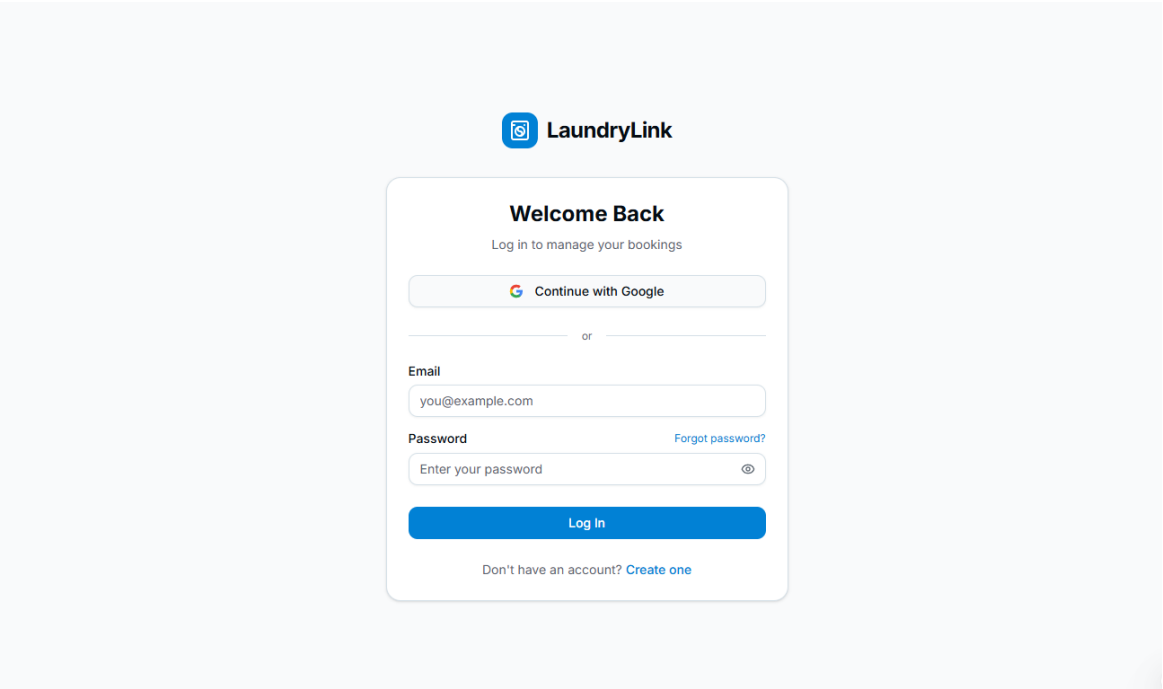
- Header: LaundryLink Logo, Login / Register button or User Menu
- Interactive Google Map embedded on the shop listing page, with a marker pin for each partner shop.
- Content: List of partner shops with name, address, and operating hours. An embedded Google Map sits above or beside the list.
- Each Shop Card: Shop info + "Book Now" button.
- Footer: Links, Copyright.

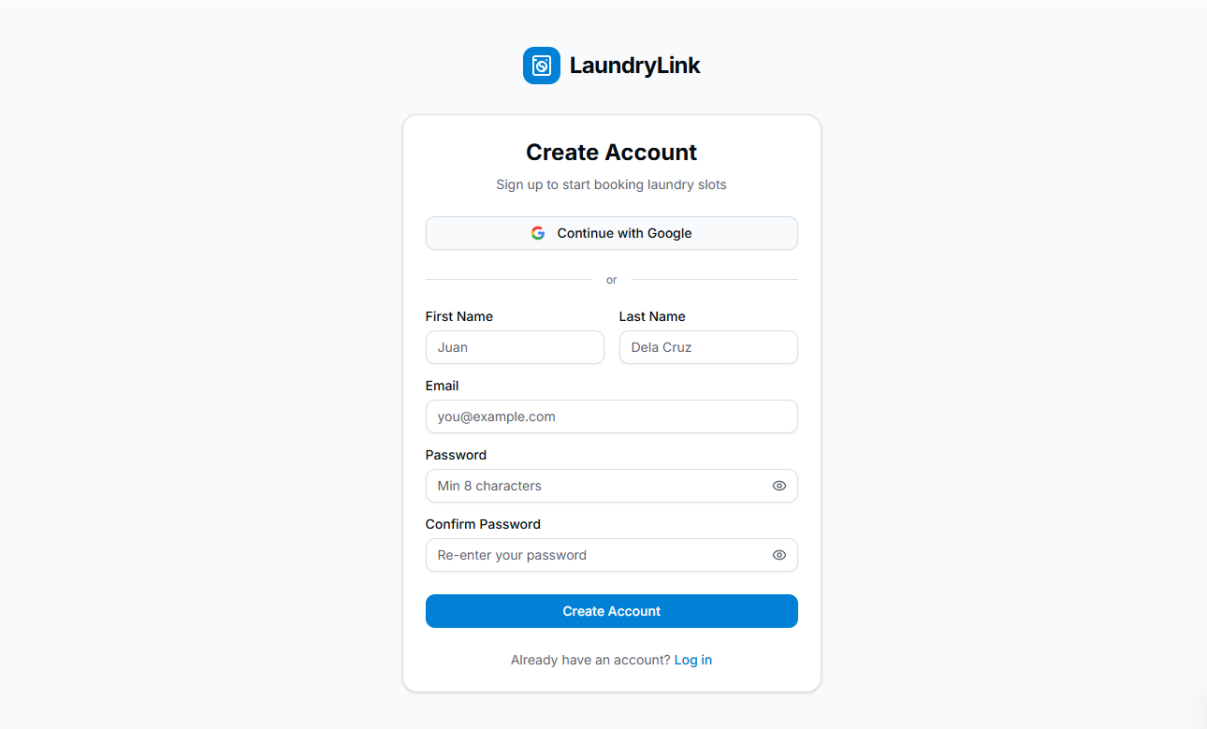

LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Partner Laundry Shops
Browse our curated list of trusted laundry shops. Select one to view services and book your priority slot.
Interactive Google Map
Shop markers will appear here with live location data
1 Priority Slot Left
FreshSpin Laundry Hub
123 Osmeña Blvd, Cebu City
7:00 AM - 9:00 PM
4.8
Standard PHP 120
Priority PHP 200
CleanWave Express
45 Mango Ave, Cebu City
6:00 AM - 10:00 PM
4.6
Standard PHP 100
Priority PHP 180
Sparkle & Fold
78 Colon St, Cebu City
8:00 AM - 8:00 PM
4.9
Standard PHP 130
Priority PHP 220
QuickDry Laundromat
200 Gorordo Ave, Cebu City
6:00 AM - 11:00 PM
4.5
Standard PHP 110
Priority PHP 190


LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Ready to Skip the Queue?
Create your free account and book your first priority laundry slot today.
Fast, predictable, hassle-free.
Get Started →
Browse Shops
LaundryLink
Prepaid priority laundry slot reservations.
Book online, pay securely, skip the queue.
Product
Find Shops
How It Works
Pricing
Account
Log In
Create Account
My Bookings
Support
Help Center
Contact Us
Privacy Policy
© 2026 LaundryLink. All rights reserved.
Built for IT342 - System Integration and Architecture

Login / Register Page

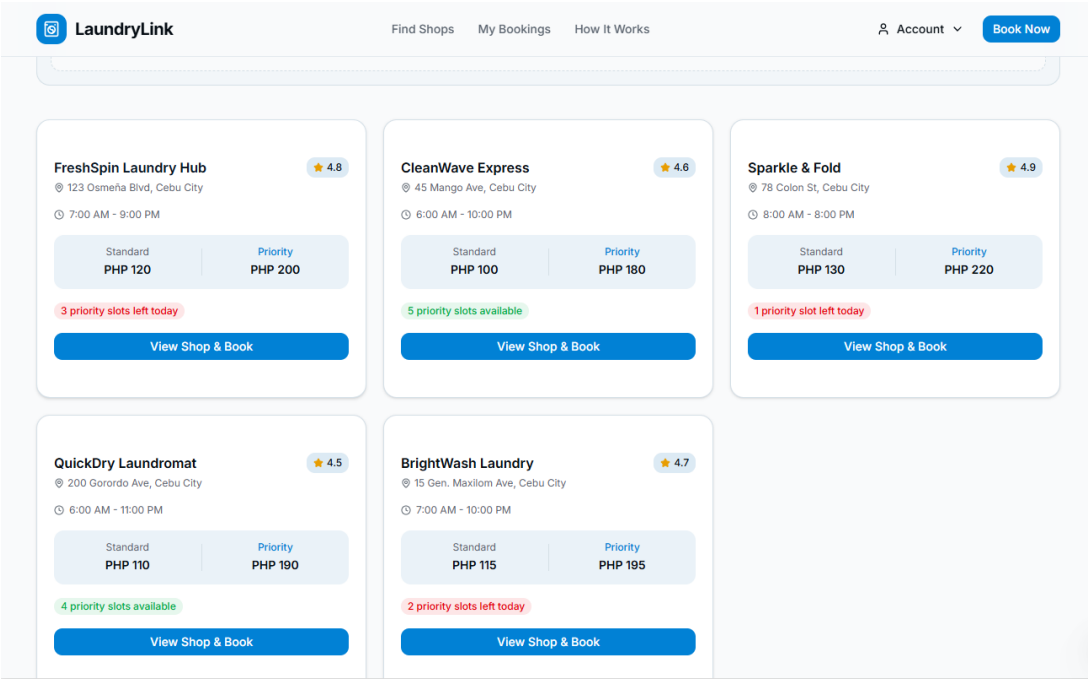
- Email/password form with validation.
- "Login with Google" button (Google OAuth).
- Link to switch between login and registration.

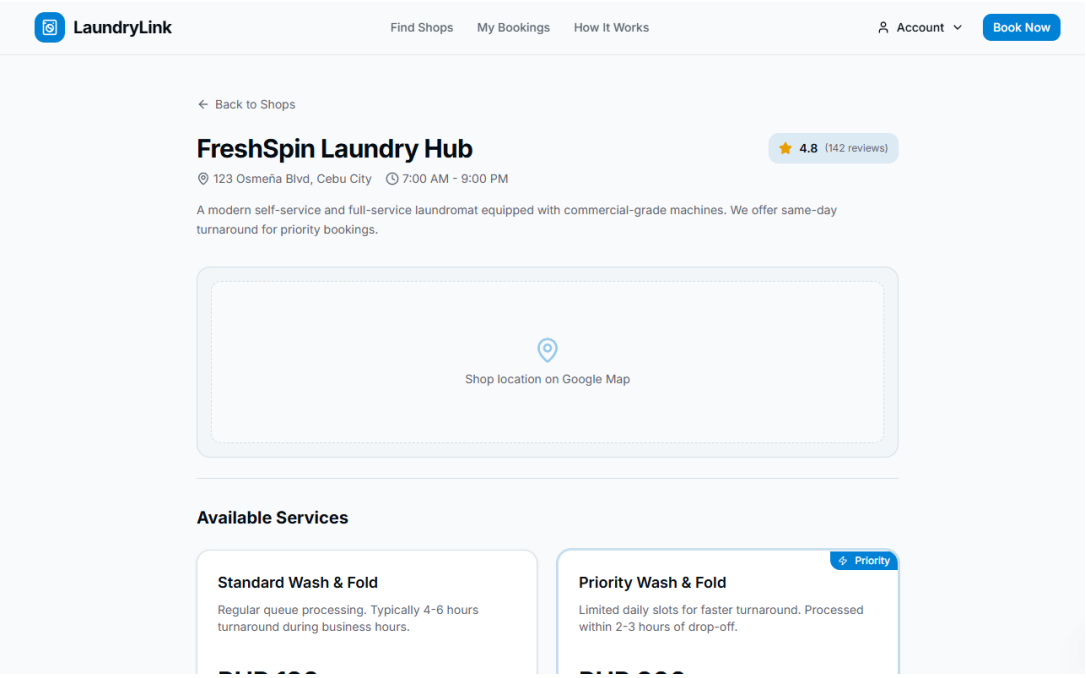


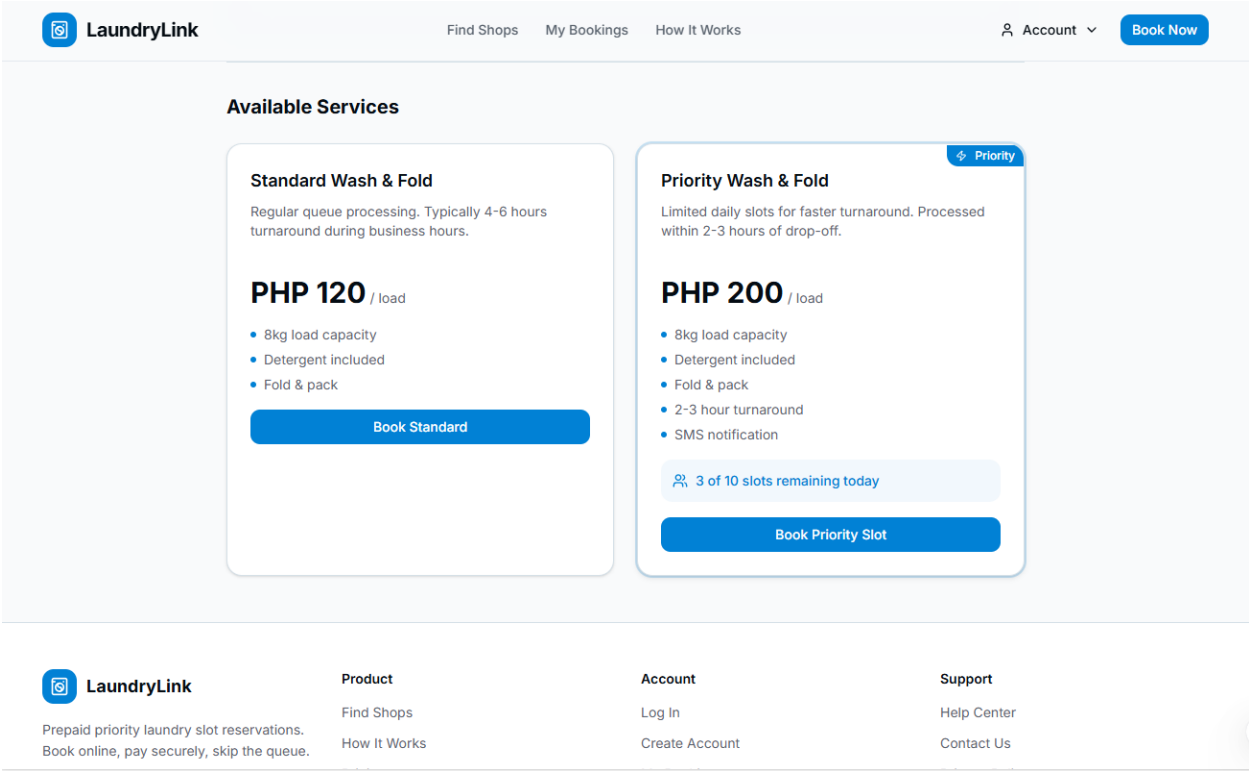


Shop Detail / Service Selection Page

- Shop Name, Address, Operating Hours.
- Service Tiers: Standard (price, queue type), Priority (price, slots remaining today).
- "Book Standard" / "Book Priority" buttons.

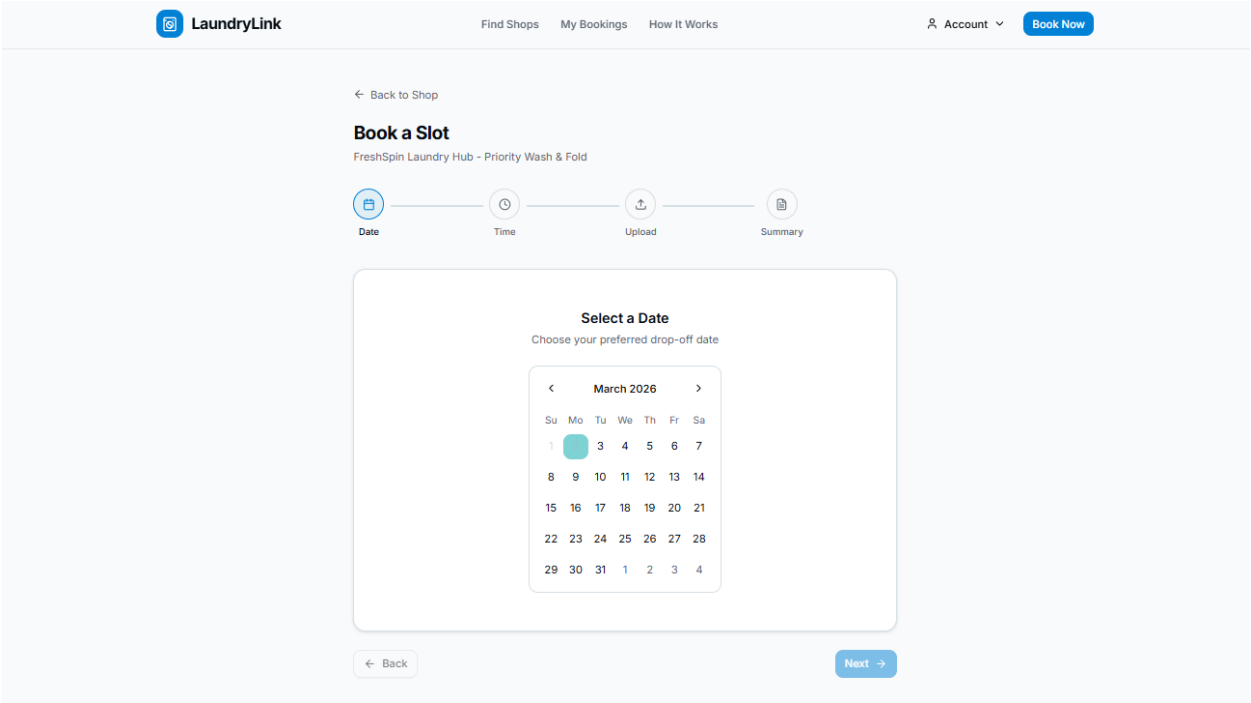


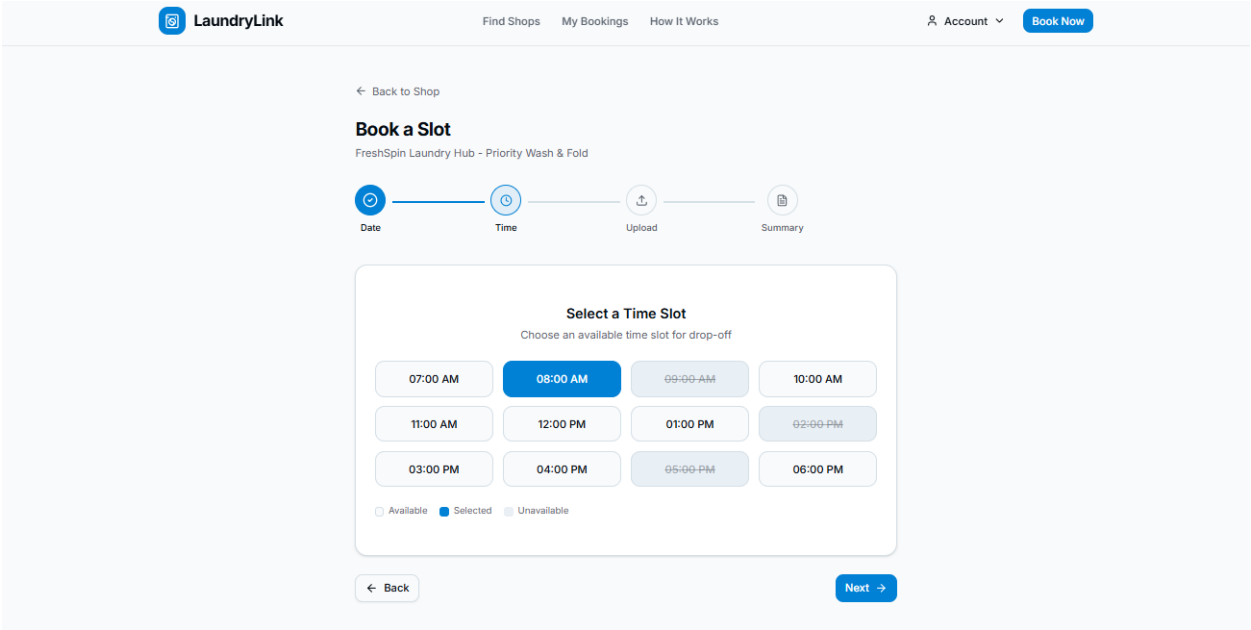


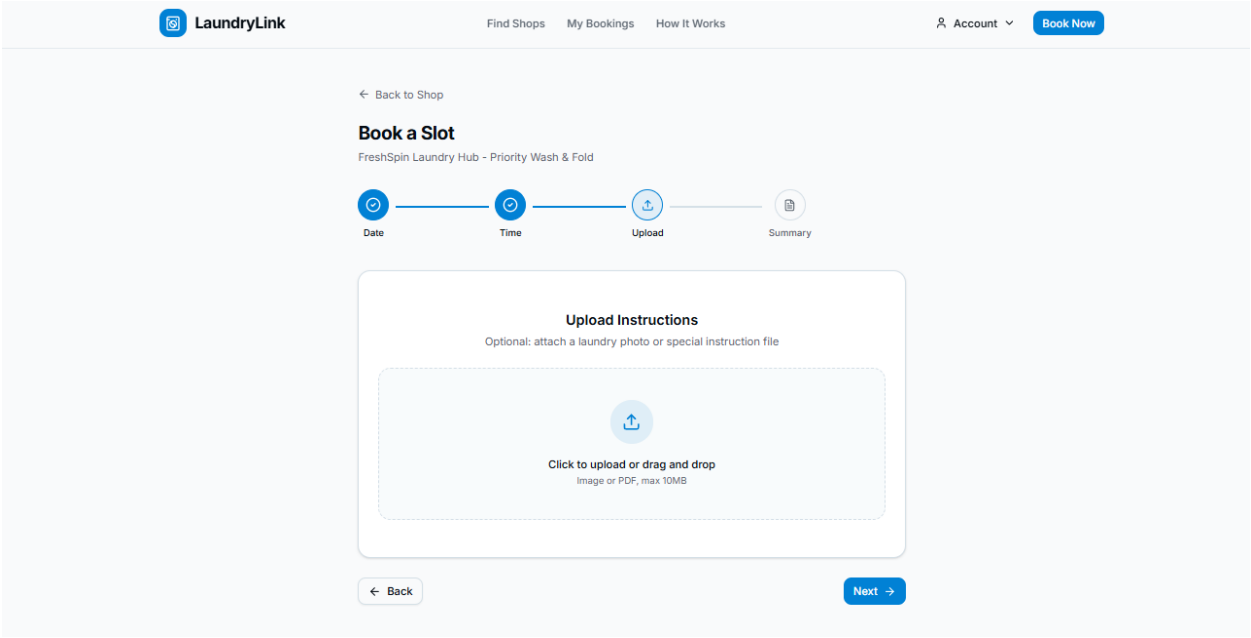


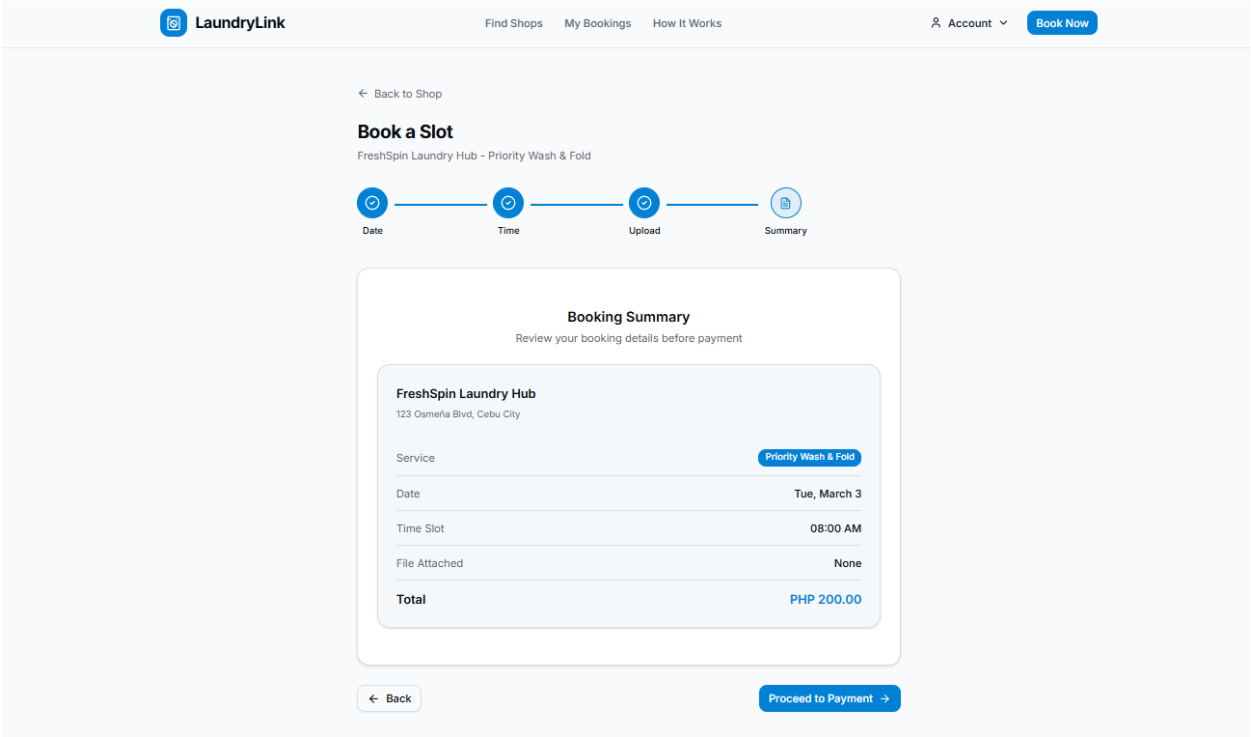
Booking Flow Page

- Step 1: Date Picker — calendar view of available dates.
- Step 2: Time Slot Selector — tap to select an available slot.
- Step 3: File Upload — optional laundry photo or PDF upload field.
- Step 4: Booking Summary — shop, service, date, time, file attached (if any), total price.
- "Proceed to Payment" button.


LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Select a Date
Choose your preferred drop-off date
March 2026
Su Mo Tu We Th Fr Sa
Back
Next

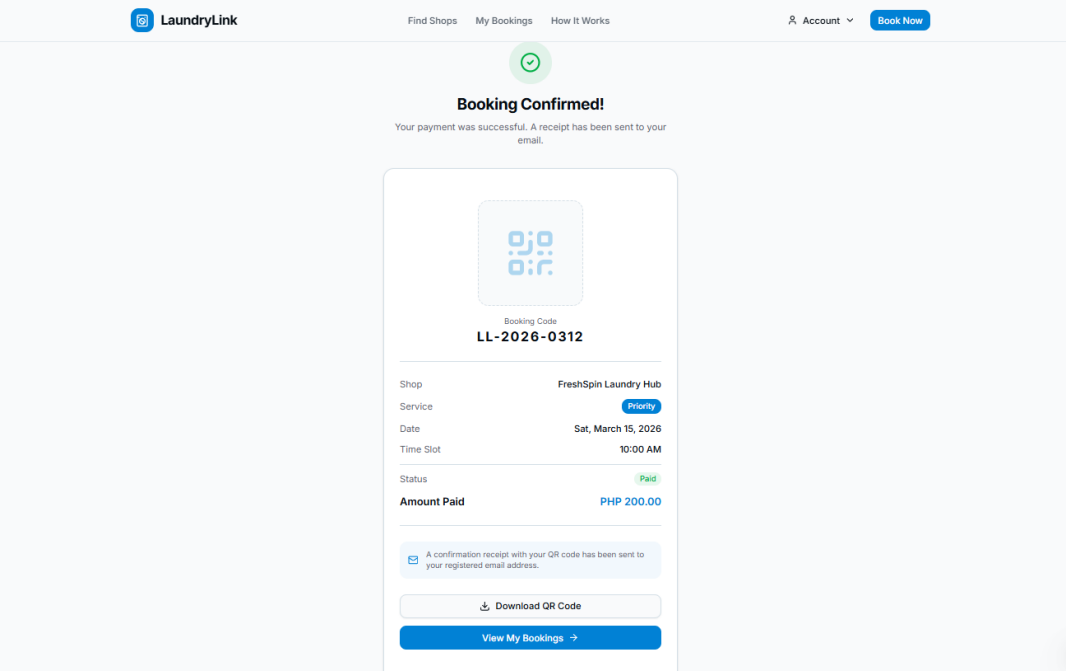

LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Select a Time Slot
Choose an available time slot for drop-off
07:00 AM
08:00 AM
09:00 AM
10:00 AM
11:00 AM
12:00 PM
01:00 PM
02:00 PM
03:00 PM
04:00 PM
05:00 PM
06:00 PM
Available
Selected
Unavailable
Back
Next


LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Upload Instructions
Optional: attach a laundry photo or special instruction file
Click to upload or drag and drop
Image or PDF, max 10MB
Back
Next


LaundryLink
Find Shops
My Bookings
How It Works
Account
Book Now
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Booking Summary
Review your booking details before payment
FreshSpin Laundry Hub
123 Osmeña Blvd, Cebu City
Service
Priority Wash & Fold
Date
Tue, March 3
Time Slot
08:00 AM
File Attached
None
Total
PHP 200.00
Back
Proceed to Payment

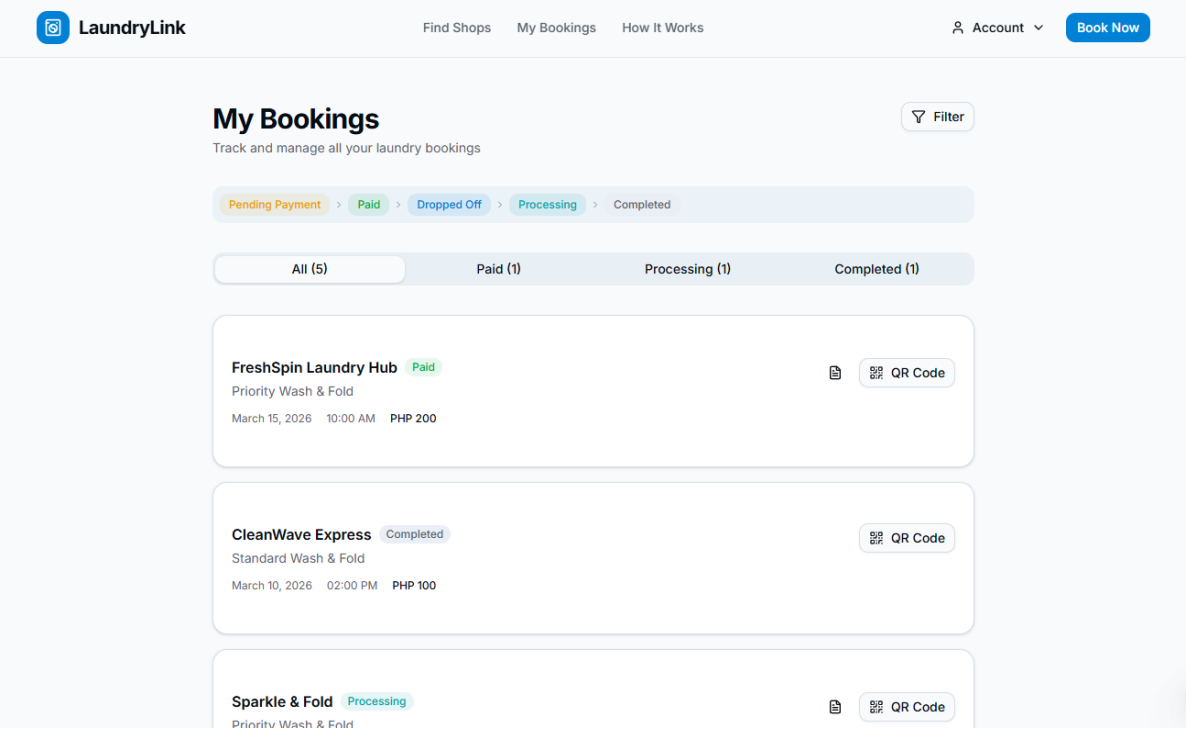
Payment & Confirmation Page

- Redirects to PayMongo-hosted sandbox payment page.
- On success: Booking Confirmation screen with Booking ID, QR Code, Shop, Date, Time, Service.
- Confirmation receipt email sent automatically.
- "Download QR" / "View My Bookings" buttons.



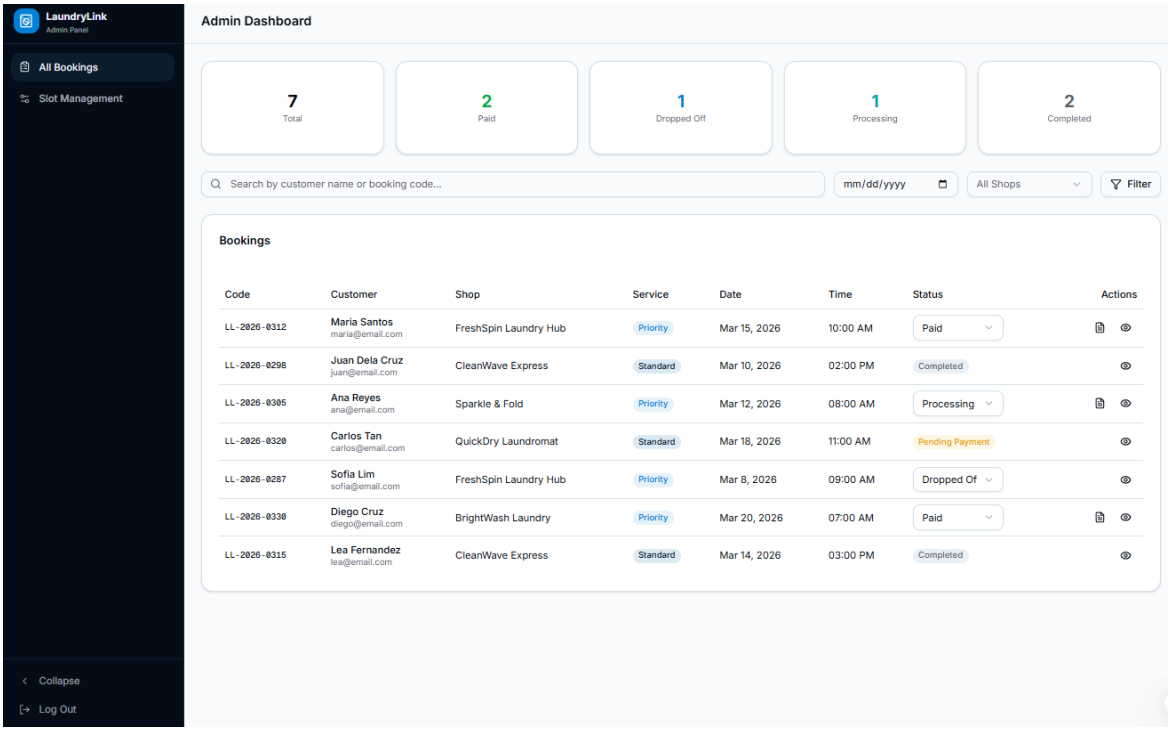
My Bookings Page (Customer)

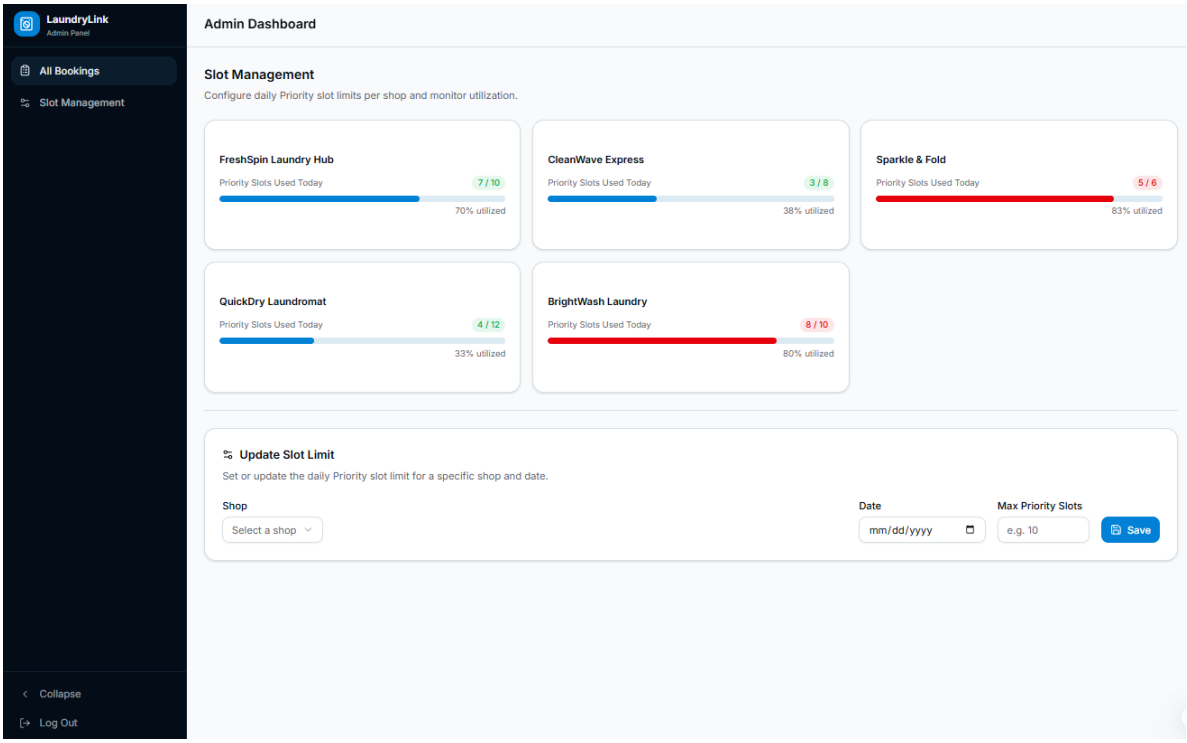
- List of all bookings with status badges.
- Status lifecycle visible: Pending Payment → Paid → Dropped Off → Processing → Completed.
- Tap booking to view QR code and view/download attached file.



Admin Dashboard

- Sidebar: All Bookings | Slot Management | Logout.
- Bookings Table: Booking ID, Customer Name, Shop, Service, Date, Time, Status, File (view icon).
- Status Update: Dropdown to change status inline per booking.
- Slot Management: Set daily Priority slot limit per shop and date.

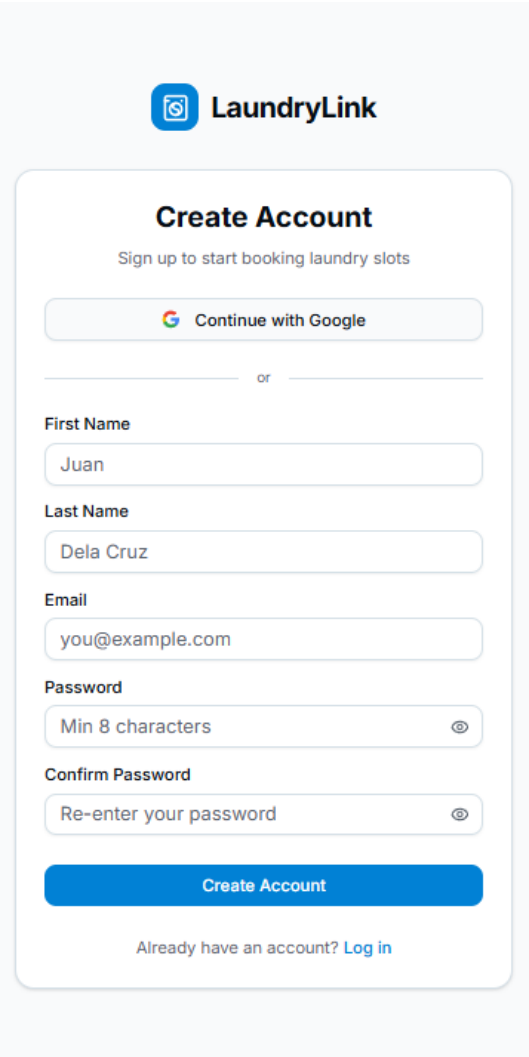


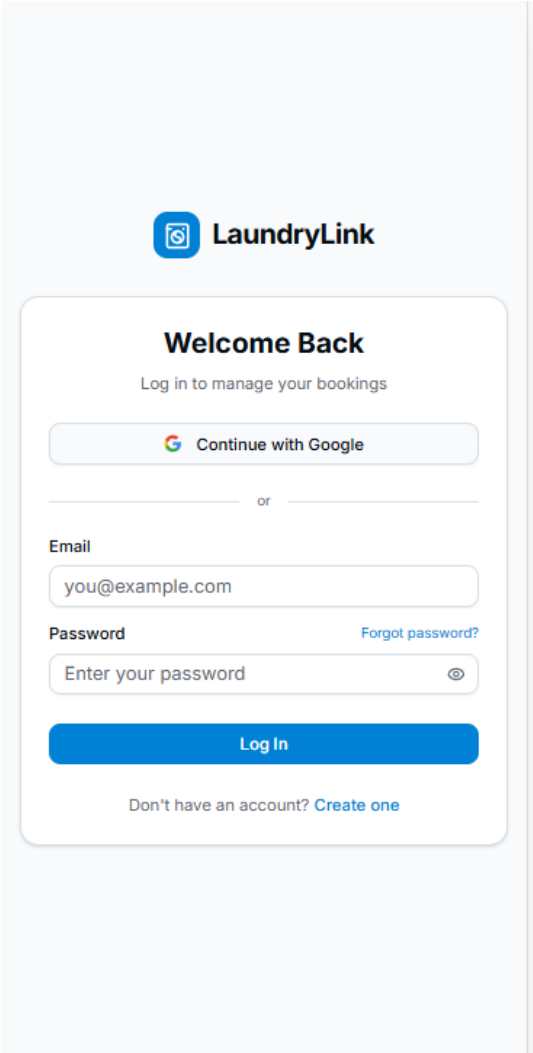

LaundryLink
Admin Panel
All Bookings
Slot Management
Collapse
Log Out
Admin Dashboard
Slot Management
Configure daily Priority slot limits per shop and monitor utilization.
FreshSpin Laundry Hub
Priority Slots Used Today
7 / 10
70% utilized
CleanWave Express
Priority Slots Used Today
3 / 8
38% utilized
Sparkle & Fold
Priority Slots Used Today
5 / 6
83% utilized
QuickDry Laundromat
Priority Slots Used Today
4 / 12
33% utilized
BrightWash Laundry
Priority Slots Used Today
8 / 10
80% utilized
Update Slot Limit
Set or update the daily Priority slot limit for a specific shop and date.
Shop
Select a shop
Date
mm/dd/yyyy
Max Priority Slots
e.g. 10
Save

7.2 Mobile Application Wireframes

https://www.figma.com/design/JmhObEMFhuAbU0rnudBQI3/LaundryLink_Mobile?node-id=0-1&t=cw8m64rdeORpvM9X-1

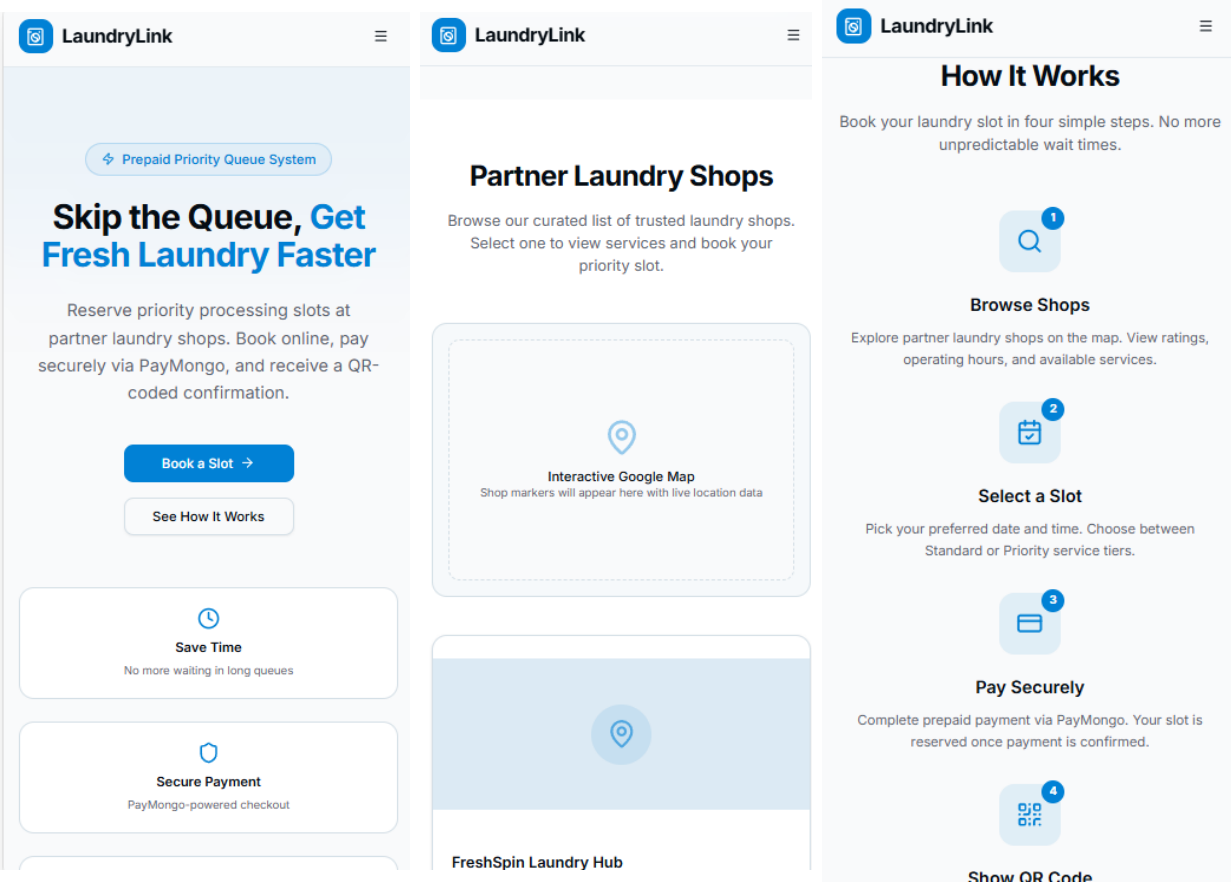
Login and Register





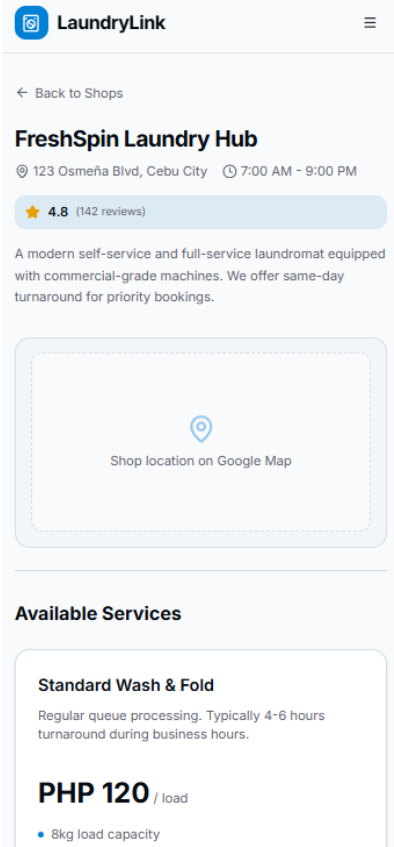
Home Screen (Shop List)

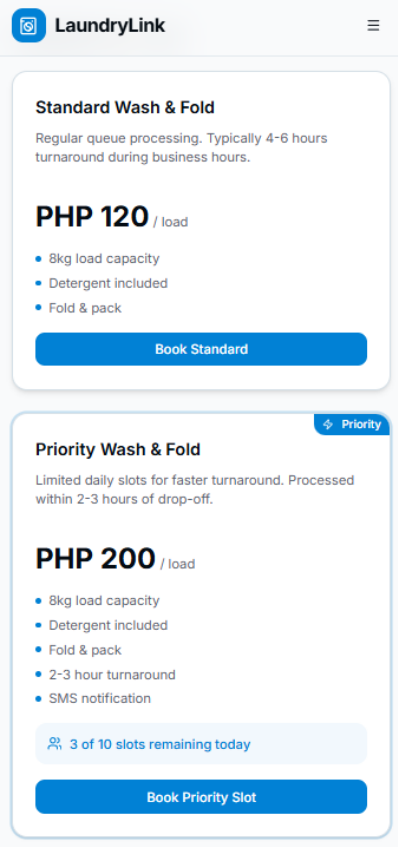
- Search/filter bar for shops.
- Embedded Google Map at the top of the screen with shop markers.
- Tap shop to view services. Pull to refresh.

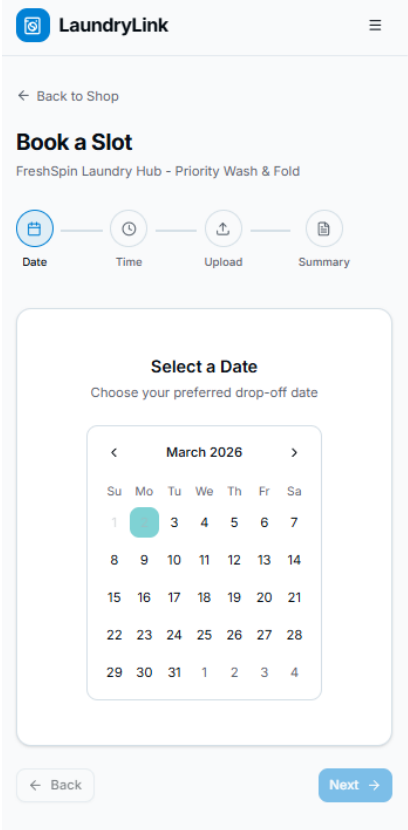


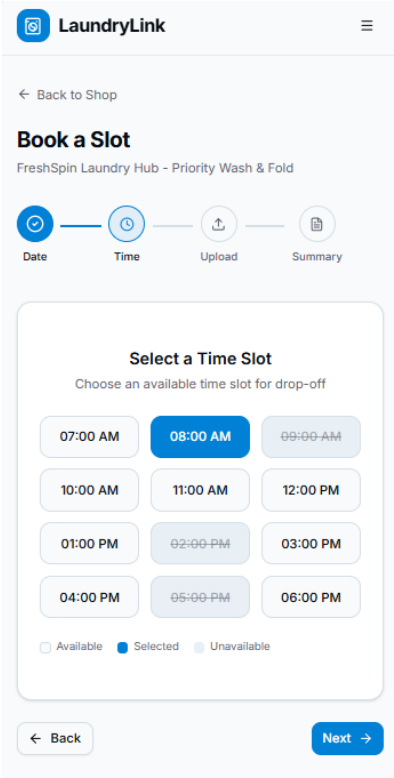
Booking Flow Screen

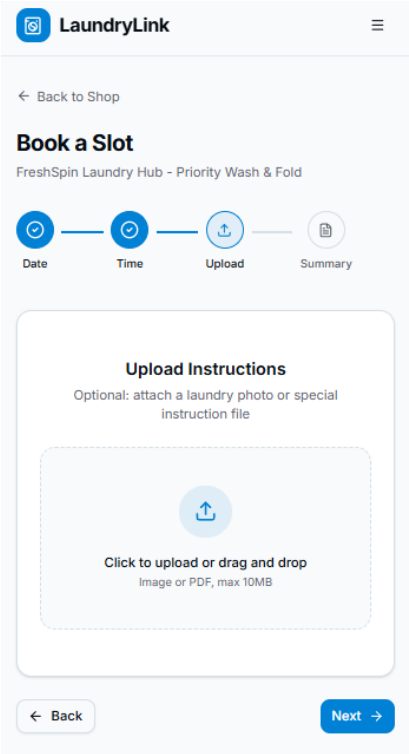
- Date picker and time slot grid.
- File attachment option (photo picker or file chooser).
- Booking summary card. "Pay Now" fixed bottom button.

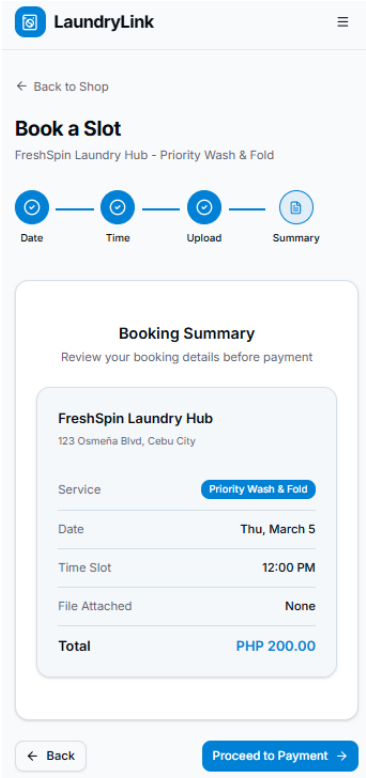

LaundryLink
Back to Shops
FreshSpin Laundry Hub
123 Osmeña Blvd, Cebu City
7:00 AM - 9:00 PM
4.8 (142 reviews)
A modern self-service and full-service laundromat equipped with commercial-grade machines. We offer same-day turnaround for priority bookings.
Shop location on Google Map
Available Services
Standard Wash & Fold
Regular queue processing. Typically 4-6 hours turnaround during business hours.
PHP 120 / load
8kg load capacity

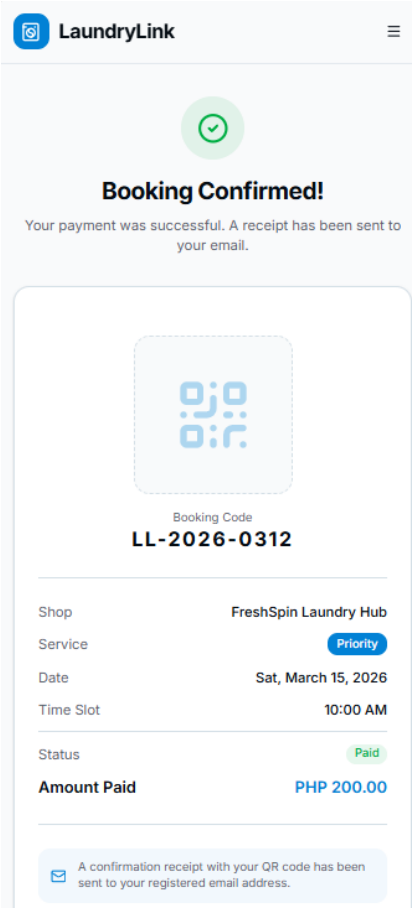

LaundryLink
Standard Wash & Fold
Regular queue processing. Typically 4-6 hours turnaround during business hours.
PHP 120 / load
8kg load capacity
Detergent included
Fold & pack
Book Standard
Priority
Priority Wash & Fold
Limited daily slots for faster turnaround. Processed within 2-3 hours of drop-off.
PHP 200 / load
8kg load capacity
Detergent included
Fold & pack
2-3 hour turnaround
SMS notification
3 of 10 slots remaining today
Book Priority Slot


LaundryLink
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Select a Date
Choose your preferred drop-off date
March 2026
Su Mo Tu We Th Fr Sa
Back
Next


LaundryLink
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Select a Time Slot
Choose an available time slot for drop-off
07:00 AM
08:00 AM
09:00 AM
10:00 AM
11:00 AM
12:00 PM
01:00 PM
02:00 PM
03:00 PM
04:00 PM
05:00 PM
06:00 PM
Available
Selected
Unavailable
Back
Next


LaundryLink
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Upload Instructions
Optional: attach a laundry photo or special instruction file
Click to upload or drag and drop
Image or PDF, max 10MB
Back
Next


LaundryLink
Back to Shop
Book a Slot
FreshSpin Laundry Hub - Priority Wash & Fold
Date
Time
Upload
Summary
Booking Summary
Review your booking details before payment
FreshSpin Laundry Hub
123 Osmeña Blvd, Cebu City
Service
Priority Wash & Fold
Date
Thu, March 5
Time Slot
12:00 PM
File Attached
None
Total
PHP 200.00
Back
Proceed to Payment

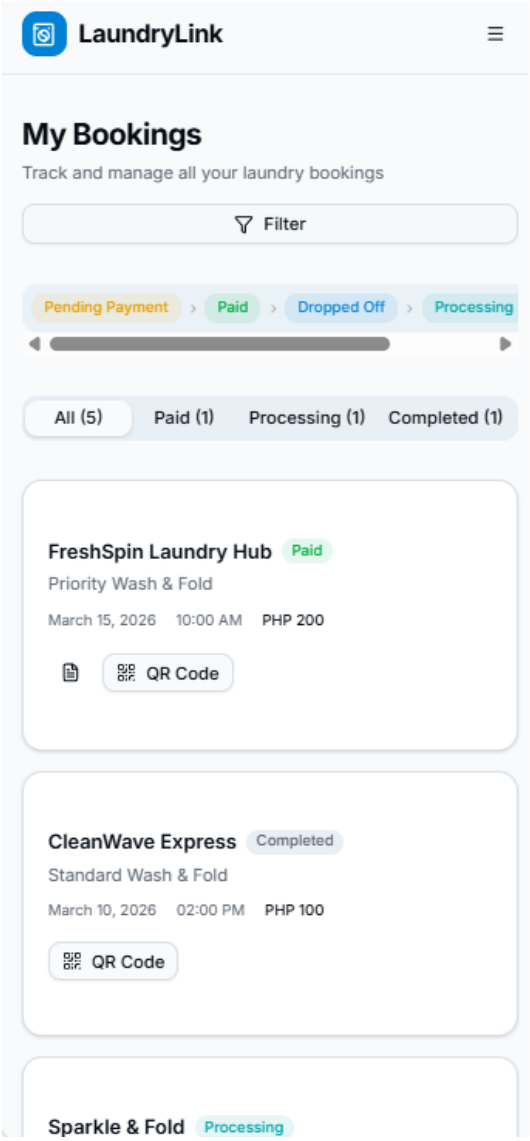


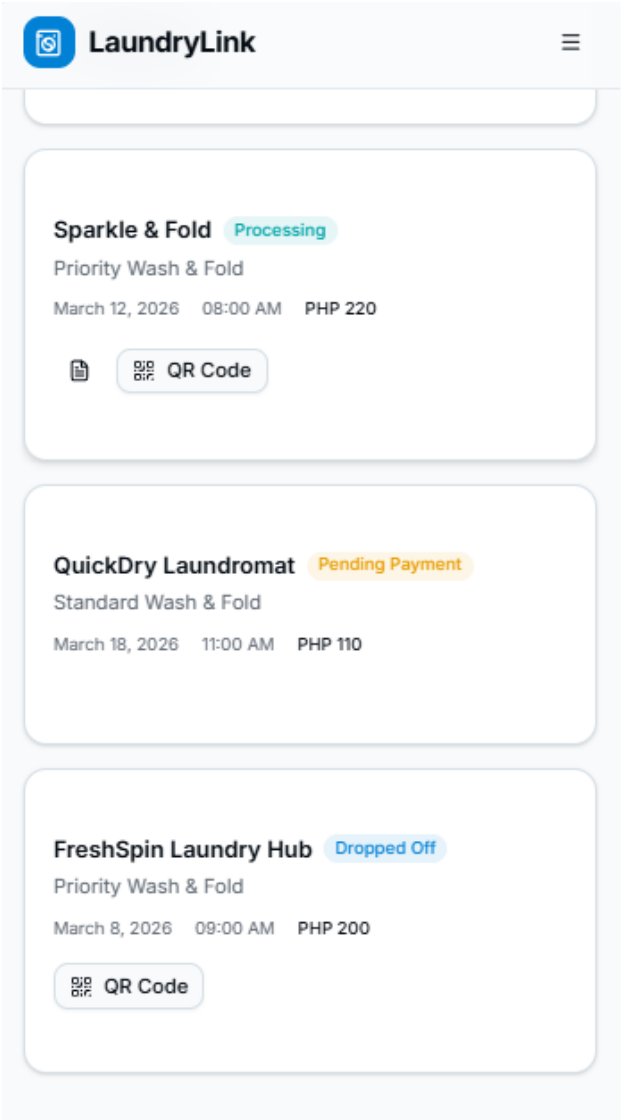
Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Gesture support (pull-to-refresh on shop and booking lists)
- QR code cached locally after booking confirmation
- Bottom navigation: Home, Bookings, Profile (customer) / Bookings, Slots (admin)
- Simplified forms with mobile-friendly date/time pickers

My Bookings Screen

- List of bookings with status badges. Filter by status.
- Tap to view booking detail, QR code, and attached file.





Design System:

Element	Specification
Colors	Primary: #3B82F6 (sky blue), Secondary: #0EA5E9, Success: #22C55E, Warning: #F59E0B, Error: #EF4444
Typography	Inter font family, responsive sizing
Spacing	8px grid system
Components	Consistent buttons, inputs, cards, modals, file upload dropzone
Responsive	Mobile-first, breakpoints at 640px, 768px, 1024px
Mobile Targets	Touch targets minimum 44×44px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning and Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup, repository initialization, SDD documentation

Day 3-4: Finalize FRS, NFR, and acceptance criteria

Day 5-7: System architecture, data flow diagrams, ERD

Week 2: Detailed Design

Day 1-2: Full API contract specification

Day 3-4: Supabase schema design and ERD

Day 5-6: Figma wireframes for web and mobile

Day 7: Finalize implementation plan and milestone tracking

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot project setup with dependencies (Spring Security, JPA, PayMongo client)

Day 2: Supabase connection, JPA entities, Flyway migrations

Day 3: JWT authentication (register, login, refresh, logout)

Day 4: Shop and service listing endpoints (seeded data)

Day 5: Slot availability check logic with capacity enforcement

Week 4: Core Features

Day 1: Booking creation endpoint with Pending Payment status

Day 2: Admin booking list and status update screen

Day 3: PayMongo webhook handler and booking status update to Paid

Day 4: QR code generation on payment confirmation

Day 5: Admin endpoints and global error handling

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: ReactJS project setup with routing and Axios

Day 2: Authentication pages (Register, Login)

Day 3: Shop listing and shop detail pages

Day 4: Booking flow (date picker, time slot selector, booking summary)

Day 5: PayMongo redirect and booking confirmation with QR display

Week 6: Complete Web Features

Day 1: My Bookings screen with status badges

Day 2: Admin booking list with date filter and status update

Day 3: Admin slot limit configuration page

Day 4: Responsive design polish and error state handling

Day 5: Full API integration and end-to-end web testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup, Kotlin project structure, Retrofit2 config

Day 2: Authentication screens (Register, Login)

Day 3: Shop list and service selection screens

Day 4: Booking flow (date/time slot picker, booking summary)

Day 5: PayMongo redirect and booking confirmation with QR display

Week 8: Complete Mobile App

Day 1: My Bookings screen with status badges

Day 2: Admin booking list and status update screen

Day 3: Admin slot limit configuration screen

Day 4: UI polish, animations, and emulator testing

Day 5: APK generation and documentation

Phase 5: Integration and Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end booking flow testing (web and mobile)

Day 2: PayMongo webhook testing with test keys

Day 3: Slot capacity enforcement edge case testing

Day 4: Admin panel full workflow testing

Day 5: Bug fixes and refinements

Week 10: Deployment

Day 1: Spring Boot deployment on Railway or Render

Day 2: ReactJS deployment on Vercel

Day 3: Android APK build and sideload distribution

Day 4: Update PayMongo webhook URL to production domain, final smoke test

Day 5: Documentation finalization and project submission

Milestones:

- **M1 (End Week 2):** All design documents and Figma wireframes complete
- **M2 (End Week 4):** Spring Boot API fully functional (auth, bookings, PayMongo webhook)
- **M3 (End Week 6):** ReactJS web application complete and integrated with backend
- **M4 (End Week 8):** Kotlin Android app complete and integrated with backend
- **M5 (End Week 10):** Full system deployed to production and project submitted

Critical Path:

1. JWT authentication system — Spring Boot (Week 3)
2. Slot availability and booking creation logic (Week 3-4)
3. PayMongo integration and webhook handler (Week 4)
4. ReactJS and Kotlin booking flow (Week 5-7)
5. End-to-end integration testing across web and mobile (Week 9)

Risk Mitigation:

- Use PayMongo test keys throughout development before switching to live.
- Test booking slot enforcement logic with unit tests early.
- Use static shop coordinate fixtures during offline development; switch to live Maps API key for integration testing.
- Test SMTP email delivery with a disposable test inbox (e.g., Mailhog or Mailtrap).
- Use Google OAuth sandbox credentials; test both new and returning user flows.
- Maintain stable Git branches as fallback at each milestone.
- Strictly enforce MVP scope — no out-of-scope features mid-development.